Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Elizabeth Davis	egdavis@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the Chempy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the Numpy and Scipy Python libraries.

I am trying to address indecision when I am bored and don't know what to do.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

- **Bored Button**: A single-click web application that routes users to a random, interactive web game or mini-experiment to instantly resolve boredom without manual filtering.
- **Goblin.tools**: An executive dysfunction toolkit with a *Magic ToDo* generator and decision manager that breaks overwhelming choices into low-friction, micro-action steps.
- **Wheel of Names / DecideNow!**: Customizable decision wheels and randomizers where users manually input candidate options and let a wheel flip or spin choose for them.
- **Llama Life**: A task-management app with a built-in **Random Task Picker** and visual timer designed to eliminate task paralysis by picking one activity at a time.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

A web application that takes user inputs (mood, budget, solo vs. group, location) and dynamically filters a database of activities to present a single recommended task.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The ultimate goal of this program is to collect four user preference variables (mood, budget, solo vs. group, location) from a web form, filter a structured dataset (list of dictionaries) of activity options matching those conditions using conditional logic, and select a single recommended activity dictionary using random choice to eliminate decision paralysis.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

- **Mood / Energy Level**: Discrete categorical choice (e.g., Low Energy, Medium Energy, High Energy).
- **Budget**: Maximum numerical dollar amount or tier selection (e.g., $0 / Free, under $15, $30+).
- **Social Setting**: Binary or dropdown choice (e.g., Solo vs. Group/Friends).
- **Location Setting**: Discrete choice (e.g., At Home, Indoor Public, Outdoor).

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The program receives user input values from the front-end interface forms. It iterates through a master list of dictionaries. It applies boolean conditional checks if statements to filter out activities exceeding the maximum cost or mismatching the mood, social setting, or location tags. From the filtered list of matching activities, the program uses Python's index-based random selector to select exactly one activity. If no activities match, a fallback handler returns a default low-friction prompt.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

Activity Title
Category / Mood Tag
Estimated Cost ($)
Quick step-by-step instructions

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

The core "thing" (nouns) my program needs to track is an Activity and User Filter Criteria.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Details I need:

- Activity Title (String)
- Required Energy/Mood (String)
- Estimated Cost in Dollars (Float or Integer)
- Group Setting Compatibility (Boolean or String: "Solo", "Group", or "Both")
- Location Type (String: "Home", "Indoor", "Outdoor")

Details I will ignore:

- Historical user reviews or star ratings
- Specific street addresses or GPS coordinates
- Exact duration down to the minute

- Required gear/equipment lists

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Filtering the activity database without crashing when no options match the user's specific inputs, and ensuring the front-end user form correctly passes data to the Python backend.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

Ensure all generated code contains inline explanatory comments. Test every filtering function and dictionary lookup in small code blocks before integrating it into the main web application to ensure I completely understand the execution flow.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

My ultimate goal for this project is to build a polished, functional web tool using Drafter that I can personally use whenever I face option paralysis, while mastering list filtering and dictionary manipulation.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours.*

What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"). Paste your prompt below:

"I am trying to address indecision when I am bored and don't know what to do. What existing tools or websites currently attempt to address it, and what core features should a simple web application include to help users pick an activity?"

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The AI highlighted tools like *Bored Button*, *Goblin.tools*, *Wheel of Names*, and *Llama Life*. It suggested key features such as single-button decision elimination, energy/mood filtering, micro-step breakdowns, and integration with real-time location/maps to recommend nearby places automatically.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

"Integrating live GPS location tracking is too complicated for an introductory CS project. How can I simplify this feature into a local Python data structure that filters options strictly using basic categorical inputs like indoor/outdoor, budget, and group size?"

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

This conversation helped narrow the project scope by eliminating complex external map APIs and location tracking. Instead, it refocused the design on using standard Python filtering logic to match four straightforward user inputs against a pre-built list of activity dictionaries.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

TA, instructors, classwork assignments on the topics, and AI to explain errors in my code.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

1. Read the error stack trace: Locate the exact line number and error type.
2. Add print statements: Insert print() statements right before the error
3. Isolate the code: Test the specific failing function in a small, separate script.
4. Ask for targeted feedback: Paste only the failing snippet and exact error message into an LLM to understand why it broke.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

User: I am trying to address indecision when I am bored and don't know what to do. What existing tools or websites currently attempt to address it, and what features could a web application include?

LLM: Boredom-induced indecision (option paralysis) is typically caused by cognitive fatigue or friction between choosing a task and starting it. Existing tools address this in a few distinct ways:

1. Bored Button: A single-click web application that routes users to a random, interactive web experiment or game to eliminate decision paralysis instantly.
2. Goblin.tools: An executive dysfunction toolkit that uses AI to break vague tasks down into manageable micro-steps to reduce starting friction.
3. Wheel of Names: A customizable decision wheel where users input candidate options and spin to let the tool make the final choice randomly.
4. Llama Life: A task-planning app with a built-in random task selector and visual timer designed to overcome task paralysis and encourage single-task focus.

Potential features for a web application:

- Taking basic user preferences (mood/energy, budget, solo vs. group, indoor vs. outdoor) to narrow down choices.

- Real-time GPS mapping and live location APIs to recommend nearby open venues automatically.
- Displaying a single activity recommendation card with an immediate action step.

User: Integrating live GPS location tracking and dynamic map APIs is too complicated for an introductory CS project. How can I simplify this to filter options using basic categorical inputs like budget, indoor/outdoor, mood, and solo/group against a simple local dataset instead?

LLM: You can simplify the project by storing all activity options locally inside your Python program as a list of dictionaries.

Data Abstraction:

- Core Entity: An Activity dictionary.
- Details Needed: title (String), mood (String), cost (Float/Int), group (String), and location (String).
- Details Ignored: Specific street addresses, user ratings, duration, or gear required.

Program Flow (IPO Model):

1. Input: Collect four categorical selections from the user form (Mood/Energy, Budget, Solo vs. Group, Indoor vs. Outdoor).
2. Processing: Loop through the list of activity dictionaries and use boolean if statements to filter for entries matching all four user criteria. Then use Python's random.choice() function to pick one match. Include a fallback if no matches are found to prevent crashes.
3. Output: Display a single activity card showing the title, matching details, and a quick starting action.

This approach keeps all logic local, eliminates complex API connections, and perfectly fits introductory programming concepts like data abstraction, conditional logic, and functions

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.